

OBJECTIVE

Build a functional AI/ML-powered pipeline that detects and classifies phishing and suspected domains targeting Critical Sector Entities (CSEs). This assignment simulates a core component of a real-world threat intelligence platform..

This assignment is intended to evaluate:

- Feature engineering and domain analysis skills
- Machine learning model design and evaluation
- Ability to work with unstructured cybersecurity data
- API design for serving ML inference results
- CLI or lightweight interface for threat insights

PROBLEM CONTEXT

Phishing attacks targeting Critical Sector Entities (CSEs) — such as banks, government portals, and infrastructure services — are a growing threat. Attackers register lookalike domains, use typosquatting, internationalized domain names (IDNs), and tunnelling services (e.g., Ngrok, Vercel) to deceive users.

Your task is to build a system that can:

- Ingest a set of domain names (phishing, suspected, and legitimate)
- Extract meaningful features from those domains
- Train and evaluate a classification model
- Expose inference results through an API
- Present findings via CLI output or a lightweight interface

DATASET

You will work with publicly available phishing domain datasets. Use any one or a combination of the following:

Source	URL / Description	Label Coverage
PhishTank	http://data.phishtank.com/data/online-valid.csv	Phishing
OpenPhish	https://openphish.com/phishing_feeds.html (free feed)	Phishing
Alexa / Tranco	Top 1M legitimate domains (tranco-list.eu)	Legitimate
DomainTools Sample	Newly registered domains from any public DNS feed	Suspected

You are also free to use your own curated dataset. The final dataset must contain a minimum of 1,000 labelled domain entries across at least two classes (Phishing and Legitimate). A third class (Suspected) is encouraged.

MANDATORY REQUIREMENT: FEATURE ENGINEERING

All model training must be based on engineered features — not raw domain strings fed directly into a neural network (unless you also implement feature-based baselines).

You must extract and use features from at least three of the following categories:

URL / Domain Structure Features

- Domain length, number of subdomains, subdomain depth
- Presence and count of hyphens, digits, special characters
- Entropy of the domain string
- Use of IP address instead of hostname
- TLD category (e.g., .com, .tk, .xyz, country-code TLDs)

Lexical Similarity Features

- Edit distance (Levenshtein) from known CSE domains
- Soundex / phonetic similarity
- Substring match with known brand names
- Presence of misleading keywords (e.g., "login", "secure", "verify")

Registration & DNS Features

- Domain age (days since registration, using WHOIS if available)
- Whether domain is newly registered (< 30 days)
- Presence of MX records
- Presence of SSL certificate

Internationalized Domain Name (IDN) Features

- Detection of homoglyph characters (e.g., a vs 4, o vs 0)
- Punycode encoding presence (xn-- prefix)
- Unicode character category distribution

Evaluation Metrics

Report the following on a held-out test set (minimum 20% split):

Metric	Why It Matters
Accuracy	Overall correctness
Precision	Minimise false positives (legit domains flagged)
Recall	Maximise true phishing domain detection
F1-Score	Balance between precision and recall
Confusion Matrix	Visual breakdown of prediction errors

Expectations

- Use scikit-learn, PyTorch — any open-source ML framework
- Implement train/validation/test split cleanly — no data leakage
- Handle class imbalance if present (SMOTE, class weights, etc.)
- Persist your trained model (joblib / pickle / ONNX)
- Save the results in a db (Mongo, SQL, etc)

BACKEND REQUIREMENTS (PYTHON + FASTAPI / FLASK)

Expose your trained model through a REST API. The API will accept domain name(s) as input and return classification results.

Minimum API Endpoints

- 1. POST /api/predict**
 - Input: { "domain": "example-login.tk" }
 - Output: label (Phishing / Suspected / Legitimate), confidence score, key features extracted
- 2. POST /api/predict/batch**
 - Input: { "domains": ["...", "..."] }
 - Output: Array of predictions with label and confidence
- 3. GET /api/stats**
 - Returns aggregate stats: total domains analysed, count per label, top risky TLDs

Expectations

- Clean project structure (routes, services, model loader)
- Input validation and error handling
- Feature extraction logic must be reusable (shared between training and inference)
- Inference should run on the saved model — not retrain on every request

DATASET HANDLING & REPRODUCIBILITY

- You are not required to commit large dataset files to your repository
- Your project must include clear instructions to download and prepare the dataset
- The training process should be reproducible by following your README
- Include a small sample dataset (min 100 rows) in your repo for quick testing
- Feature extraction code must be independent of dataset format changes

EVALUATION CRITERIA

Area	What We Look For	Weight
Feature Engineering	Quality, variety, and domain relevance of extracted features	High
Model Design	Appropriate model choice, evaluation rigour, handling of imbalance	High
API Design	Endpoint clarity, error handling, inference pipeline quality	Medium
Code Quality	Organisation, readability, modularity	Medium
README / Documentation	Setup clarity, design decisions, assumptions stated	Medium

BONUS (OPTIONAL)

- Implement IDN / homoglyph detection (e.g., flag xn-- encoded domains or mixed-script domains)
- Add a domain monitoring stub — given a list of suspected domains, re-evaluate them on a schedule
- Dockerize the application (API)
- Deploy the API (Render, Railway, Hugging Face Spaces, etc.)
- SHAP or LIME feature importance visualisation to explain model predictions
- Experiment tracking with MLflow or Weights & Biases
- Implement a simple Whois/DNS lookup fallback for domains without pre-extracted features

SUBMISSION REQUIREMENTS

- All submissions must be made through the following Google Form:
 - <https://docs.google.com/forms/d/e/1FAIpQLSeXsHyu5xDRBth2IVbPqQNpuNeIDQS0gpa-ITju95oKq5xM3w/viewform?usp=publish-editor>
- Do not send your assignment via email. Submissions shared over email will not be considered.

Ensure your GitHub repository is publicly accessible and includes:

- Setup and installation instructions
- Steps to download and prepare the dataset
- How to train the model (command or notebook)
- How to start the API server
- API documentation (endpoint list, request/response schema)
- Model performance summary (metrics table)
- Design decisions and assumptions

Ensure your resume is shared via a publicly accessible link (Google Drive or similar).

Before submitting, verify that:

- All links are accessible without requiring permissions
- The project can be set up and run using your README instructions
- The saved model file is included in the repository or downloadable via a script